

Estructuras de Control

IF y CASE permiten que el procedimiento tome decisiones según los datos. **IF** evalúa condiciones booleanas; **CASE** selecciona entre múltiples opciones de forma más estructurada.

FOR y **WHILE** se utilizan como estructuras cíclicas.

IF ... THEN ... ELSE

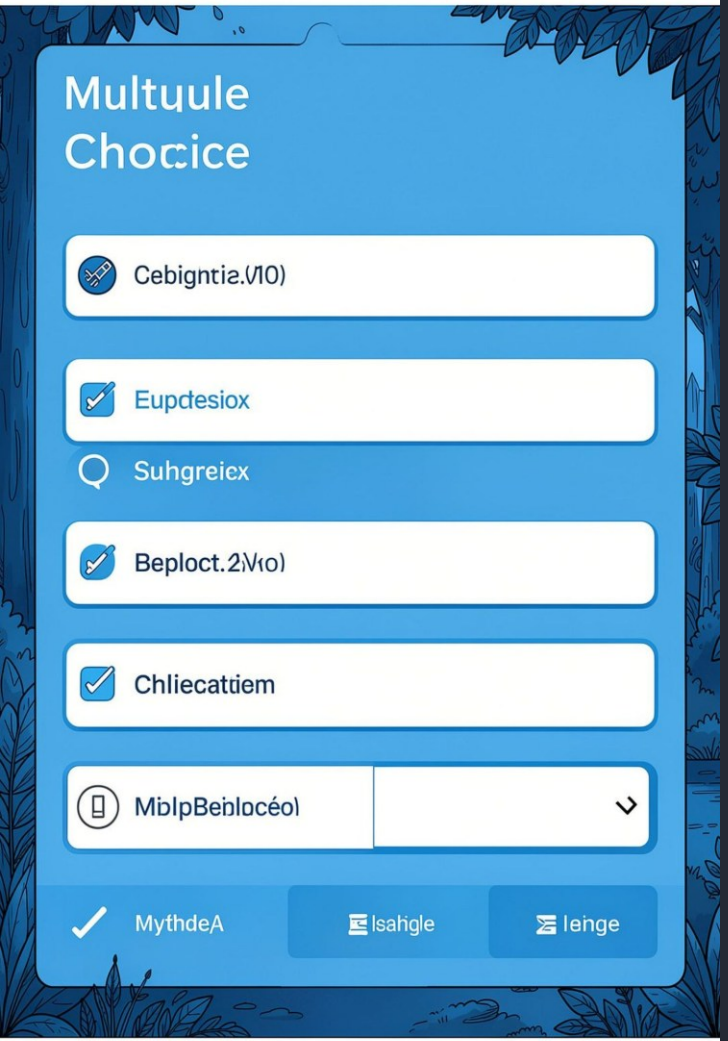
Sintaxis completa

```
IF condicion THEN
    -- acciones si es TRUE
ELSIF otra_condicion THEN
    -- acciones si es TRUE
ELSE
    -- acciones si ninguna es TRUE
END IF;
```

Ejemplo: cálculo de bonus

```
IF salario > 10000 THEN
    bonus := salario * 0.10;
ELSIF salario > 5000 THEN
    bonus := salario * 0.05;
ELSE
    bonus := 0;
END IF;
```

ELSIF (sin E final) es la forma correcta en PL/pgSQL, no ELSEIF.



Estructura CASE

Dos variantes: **CASE variable** (compara un valor concreto) y **CASE WHEN condición** (más flexible, permite rangos y expresiones complejas).

```
CASE
 WHEN x BETWEEN 0 AND 10 THEN msg := 'Entre 0 y 10';
 WHEN x BETWEEN 11 AND 20 THEN msg := 'Entre 11 y 20';
 ELSE msg := 'Otro valor';
END CASE;
```

Estructuras Cíclicas

LOOP (bucle básico)

Bucle infinito controlado con `EXIT WHEN`. Útil cuando la condición de salida no se conoce al inicio.

WHILE (evaluación previa)

Evalúa la condición **antes** de cada iteración. Si es falsa desde el inicio, el cuerpo no se ejecuta nunca.

FOR numérico

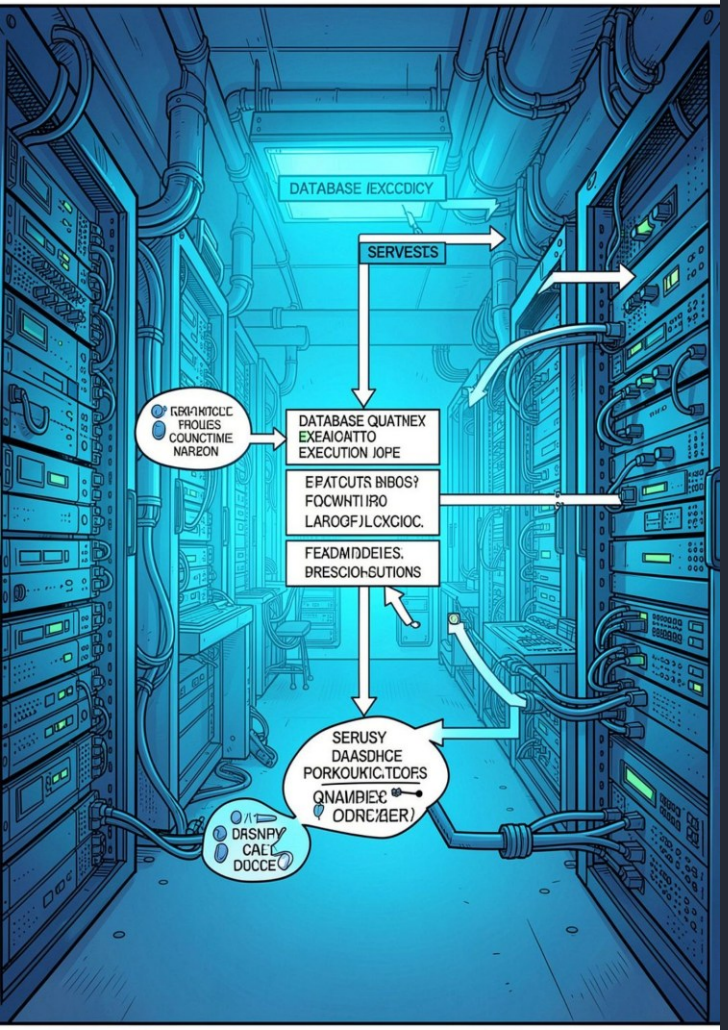
Itera sobre un rango definido. Soporta `REVERSE` para recorrer en orden descendente.

FOR sobre query

Recorre automáticamente cada fila devuelta por un `SELECT`. Ideal para procesar resultados de tablas.

```
FOR i IN 1..10 LOOP
 RAISE NOTICE 'Valor: %', i;
END LOOP;

FOR emp IN SELECT * FROM empleados LOOP
 RAISE NOTICE 'Empleado: %', emp.nombre;
END LOOP;
```



Sentencias SQL en el SP

SELECT INTO

Almacena el resultado de una consulta en variables. Debe devolver exactamente una fila.

PERFORM

Ejecuta una consulta y descarta el resultado. Útil para efectos secundarios o verificar existencia con `FOUND`.

RETURNING INTO

Captura valores generados por `INSERT`, `UPDATE` o `DELETE` (ej: IDs autoincrementales).

Sentencias SQL en el SP

SELECT INTO

```
CREATE FUNCTION obtener_empleado(p_id integer) RETURNS text AS
$$
DECLARE
    v_nombre  text;
    v_salario numeric;
BEGIN
    SELECT nombre, salario
    INTO v_nombre, v_salario
    FROM empleados
     WHERE id = p_id;
    RETURN v_nombre || ' cobra ' || v_salario;END;$$ LANGUAGE plpgsql;
END;$$ LANGUAGE plpgsql;
```

RETURNING INTO

```
 INSERT INTO empleados(nombre, salario)
VALUES (p_nombre, p_salario)
RETURNING id INTO v_id;
```

```
 PERFORM registrar_acceso(25);
-- No importa el resultado
```

Variable Especial: FOUND

¿Qué es?

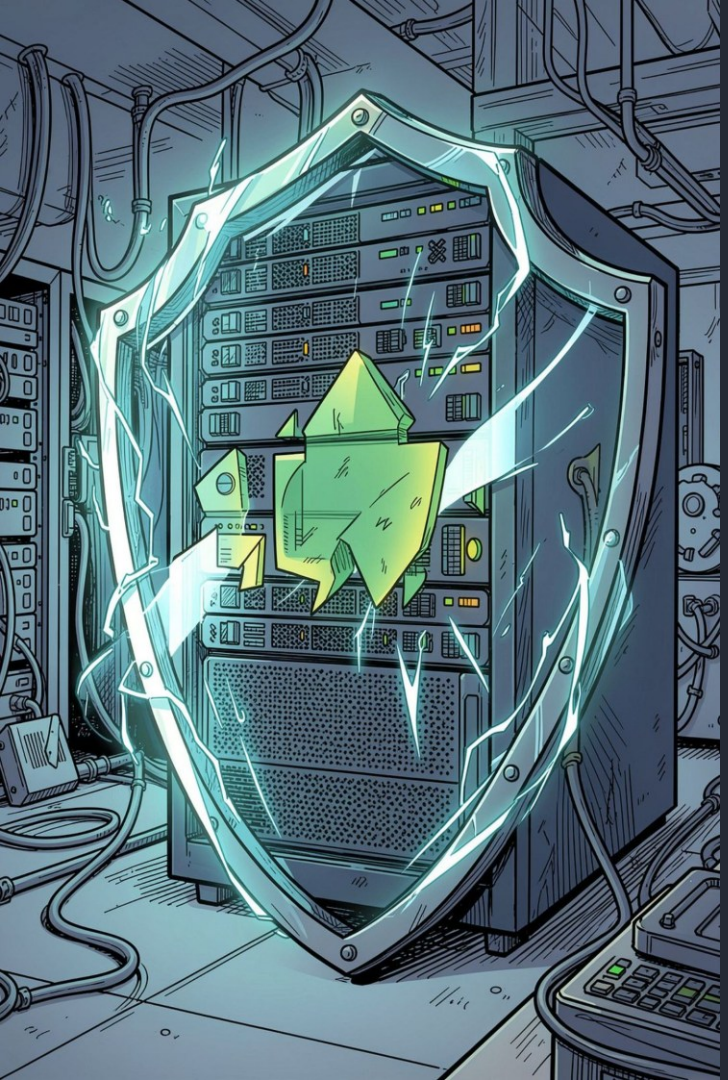
Variable `BOOLEAN` que se actualiza automáticamente tras cada operación SQL. Indica si la operación afectó o devolvió al menos una fila.

- **SELECT INTO**: TRUE si se asignó al menos una fila
- **PERFORM**: TRUE si la consulta devolvió filas
- **UPDATE / INSERT / DELETE**: TRUE si afectó filas
- **FOR**: TRUE si el bucle iteró al menos una vez

Ejemplo de uso

```
SELECT * INTO v_reg
FROM productos
WHERE id = p_id;

IF NOT FOUND THEN
 RAISE EXCEPTION
 'Producto % no encontrado',
 p_id;
END IF;
```



Manejo de Excepciones

Permite capturar errores sin interrumpir la ejecución. Sintaxis: `BEGIN ... EXCEPTION ... END;`

```
BEGIN
    INSERT INTO logs (mensaje) VALUES (v_texto);
EXCEPTION
    WHEN unique_violation THEN
        RAISE NOTICE 'Log duplicado, ignorando...';
        RETURN NULL;
    WHEN OTHERS THEN
        RAISE WARNING 'Error inesperado: %', SQLERRM;
END;
```

unique_violation

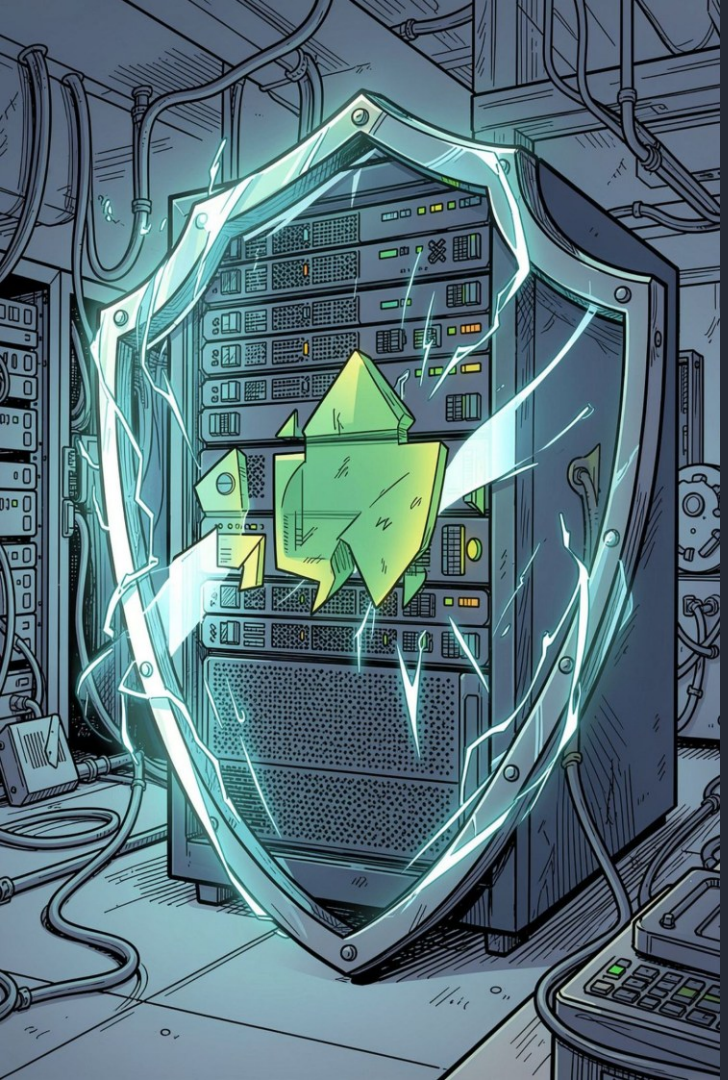
Código 23505

foreign_key_violation

Código 23503

invalid_text_representation

Código 22P02



Manejo de Excepciones

Otro ejemplo:

```
BEGIN;  -- transacción exterior

    INSERT INTO logs VALUES (100, 'X');  -- ✓

    BEGIN
      INSERT INTO logs VALUES (1, 'A');  -- ✓ (se revierte)
      INSERT INTO logs VALUES (1, 'B');  -- ✗
      INSERT INTO logs VALUES (2, 'C');  -- no ejecuta
    EXCEPTION
      WHEN unique_violation THEN
        RAISE NOTICE 'Error';
    END;

    INSERT INTO logs VALUES (200, 'Y');  -- ✓

    COMMIT;
```

SQL Dinámico con EXECUTE

¿Cuándo usarlo?

Para construir consultas en tiempo de ejecución: nombres de tablas variables, filtros dinámicos o estructuras que no se conocen al compilar.

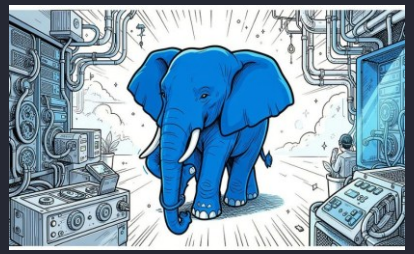
⚠ Siempre usá `quote_ident()` y `USING` para evitar inyección SQL.

Ejemplo seguro

```
EXECUTE 'SELECT * FROM '
 || quote_ident(nombre_tabla)
 || ' WHERE id = $1'
INTO v_resultado
USING p_id;

-- Con format()
EXECUTE format(
 'UPDATE %I SET %I = $1 WHERE id = $2',
 v_tabla, v_columna
) USING v_valor, v_id;
```

Lenguajes PL Soportados



PL/pgSQL

Lenguaje estándar de PostgreSQL. Visto a lo largo del curso.



PL/Python

Ideal para lógica compleja, expresiones regulares y estadísticas. Requiere `CREATE EXTENSION plpython3u;`



PL/Perl y PL/Tcl

Alternativas scripting para casos de uso específicos.



C

Lenguaje compilado externo para máximo rendimiento.

Ejemplos en PL/Python

Función simple: longitudes de palabras

```
CREATE FUNCTION longitudes_palabras(texto TEXT)
RETURNS SETOF INTEGER AS $$
    palabras = texto.split(',')
    for p in palabras:
        yield len(p.strip())
$$ LANGUAGE plpython3u;

-- Invocación:
SELECT * FROM longitudes_palabras('Hola,Mundo,SQL');
-- Devuelve: 4, 5, 3
```

Procesamiento por lotes con cursor

```
CREATE FUNCTION contar_impares_batch(batch_size INT)
RETURNS INTEGER AS $$
 odd = 0
 cursor = plpy.cursor("SELECT num FROM tabla_grande")
 while True:
 rows = cursor.fetch(batch_size)
 if not rows: break
 for r in rows:
 if r['num'] % 2 == 1:
 odd += 1
 return odd
$$ LANGUAGE plpython3u;
```

